

Production Engineering Playbook for Microsoft Foundry Agentic AI Applications

A study guide for developing, deploying, evaluating, and operating production-grade agentic AI systems on Microsoft Foundry

Audience: Engineers and architects who already understand basic agentic AI and want to build real enterprise systems using Microsoft Foundry, Microsoft Agent Framework, Azure AI Search, and Azure observability services.

Updated: May 6, 2026. Microsoft AI services and SDKs evolve quickly; verify package versions and API signatures before production use.

Core outcome: by the end of this guide, you should be able to design, build, test, deploy, observe, secure, and continuously improve a production-grade agentic AI application.

Table of Contents

- 1. Production mindset and mental model
- 2. Microsoft Foundry ecosystem
- 3. Expert maturity model
- 4. Reference architecture
- 5. Environment setup
- 6. First Agent Framework agent
- 7. Tool design and human approval
- 8. RAG and grounding with Azure AI Search
- 9. Workflows and multi-agent design
- 10. Middleware, guardrails, and structured output
- 11. Evaluation and GenAIOps
- 12. Observability and tracing
- 13. Security and responsible AI
- 14. Deployment and operations
- 15. Capstone project
- 16. 12-week plan
- 17. Q&A bank
- 18. References

1. Production mindset and mental model

A production agentic AI application is not just a chatbot with tools. It is a software system where probabilistic reasoning, deterministic code, identity, data access, evaluation, monitoring, and governance all work together.

Airport analogy	Agentic AI equivalent
Passenger	User request
Pilot	Agent
Air traffic control	Orchestrator or workflow
Flight plan	Prompt, plan, and task state
Radar	Tracing and observability
Passport control	Identity and authorization
Ground crew	Tools and APIs
Maintenance checks	Evaluation and regression tests
Emergency protocol	Fallback, rollback, and human escalation

Expert mindset: Do not ask only whether the agent can answer. Ask whether you can prove it answers correctly, safely, securely, observably, repeatedly, and within cost limits.

2. Microsoft Foundry ecosystem

Microsoft Foundry Agent Service is a managed platform for building, deploying, and scaling agents. Foundry Agent Service uses agents, conversations, and responses as runtime components. An agent combines a model, instructions, and tools; a conversation persists multi-turn history; a response is generated from user input and agent execution.

Microsoft Agent Framework is the code-first framework for building AI agents and multi-agent workflows in Python and .NET. Use it when you need explicit control over orchestration, tools, middleware, workflows, and production engineering patterns.

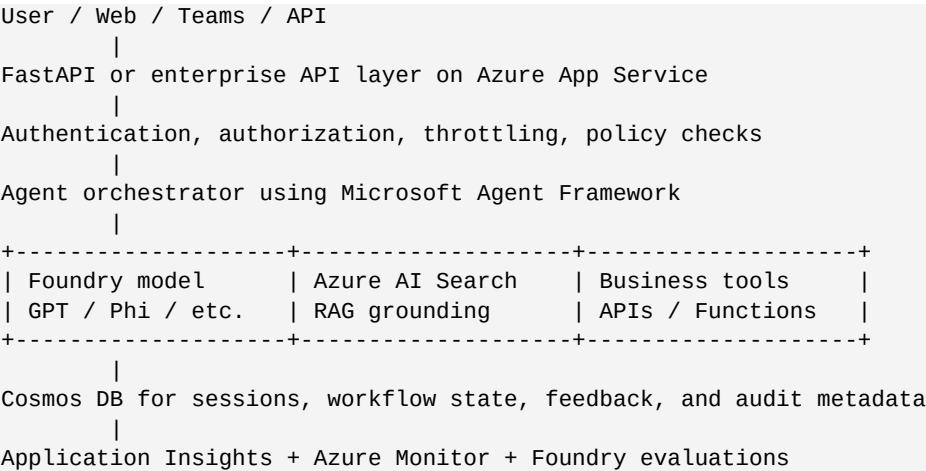
Component	Primary use
Microsoft Foundry	Platform for projects, models, agents, evaluation, governance, connections, and deployment lifecycle.
Foundry Agent Service	Managed agent runtime with agents, conversations, responses, tools, and hosted deployment options.
Microsoft Agent Framework	Code-first framework for agents, workflows, tools, middleware, and multi-agent systems.
Azure AI Search	Retrieval and grounding layer for enterprise RAG.
Azure App Service / Container Apps / AKS	Hosting layer for APIs and agent applications.
Cosmos DB	Conversation state, workflow state, feedback, audit metadata, and operational records.
Application Insights / Azure Monitor	Traces, metrics, logs, alerts, latency, failures, and operational dashboards.

When to use both: Use Agent Framework for code-first engineering and Foundry for model access, managed agent lifecycle, deployment, evaluation, governance, and operations.

3. Expert maturity model

Level	Capability	What changes
1. Prompt app	Single prompt, no tools, no RAG	Useful demo but not production ready.
2. Tool-using agent	Typed tools and basic logs	Agent can interact with systems safely in narrow ways.
3. Grounded enterprise agent	RAG, citations, access control trimming	Answers use trusted sources rather than model memory.
4. Production workflow	Explicit workflows, eval gates, tracing, fallbacks	System becomes testable, observable, and recoverable.
5. Governed platform	Versioning, red-team tests, SLOs, cost governance, incident playbooks	Enterprise-ready operating model.

4. Reference architecture



API layer rule: Do not let the UI call the model or agent service directly in most enterprise apps. Put an API layer in front to enforce identity, authorization, rate limits, logging, and policy.

5. Environment setup

Use Python 3.10 or later. Keep package installation explicit. The broad package is convenient for learning; the Foundry-focused package is better for smaller production deployments.

```
python -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip

# Broad local development install
pip install agent-framework azure-identity

# Lightweight Foundry-focused install
pip install agent-framework-foundry azure-identity

# Azure AI Foundry project and agent service operations
pip install "azure-ai-projects>=2.0.0" azure-identity
```

```
export AZURE_AI_PROJECT_ENDPOINT="https://<your-project>.cognitiveservices.azure.com"
export AZURE_AI_MODEL_DEPLOYMENT_NAME="gpt-4o"
```

Version warning: Microsoft Agent Framework and Foundry SDKs change quickly. Pin and test versions in production, and verify API signatures before copying sample code into release branches.

6. First Agent Framework agent

An agent is like a smart employee with a job description. The model is the brain, the instructions are the job description, the tools are the systems it can access, and the workflow is the operating procedure.

```
import asyncio
import os

from azure.identity.ai import DefaultAzureCredential
from agent_framework_azure_ai import AzureAIAgentClient

async def main() -> None:
    credential = DefaultAzureCredential()

    client = AzureAIAgentClient(
        project_endpoint=os.environ["AZURE_AI_PROJECT_ENDPOINT"],
        model_deployment_name=os.environ["AZURE_AI_MODEL_DEPLOYMENT_NAME"],
        credential=credential,
    )

    agent = client.create_agent(
        name="production-study-agent",
        instructions=(
            "You are a senior Azure AI architect. "
            "Explain concepts clearly and prefer secure, observable, "
            "production-ready designs."
        ),
    )
```

```

    response = await agent.run(
        "Explain the production difference between an assistant, an agent, and a workflow."
    )
    print(response.text)

    await client.close()
    await credential.close()

if __name__ == "__main__":
    asyncio.run(main())

```

Q: Should every problem use an agent?

A: No. If a deterministic function solves the task, use a function. Agents are valuable when natural-language reasoning, tool choice, uncertainty handling, or multi-step planning are needed.

7. Tool design and human approval

A tool is like giving an employee access to a company system. Tool design must be narrow, typed, validated, authenticated, authorized, logged, and auditable.

```

from typing import Annotated
from agent_framework import ai_function

@ai_function(
    name="get_order_status",
    description="Get the current status of a customer order by order ID.",
)
async def get_order_status(
    order_id: Annotated[str, "The customer order ID, for example ORD-10023"]
) -> dict:
    # In production, call your real API here.
    orders = {
        "ORD-10023": {"status": "shipped", "eta": "2026-05-09"},
        "ORD-10024": {"status": "processing", "eta": "2026-05-11"},
    }
    return orders.get(order_id, {"status": "not_found", "eta": None})

```

Production tool checklist:

- ☐ Clear name and description
- ☐ Typed parameters
- ☐ Input validation
- ☐ Authentication and authorization
- ☐ Timeout and retry policy
- ☐ Error handling
- ☐ Audit logging
- ☐ Idempotency for state-changing actions
- ☐ Human approval for high-risk actions

High-risk tools should require approval. Examples include refunds, deletion, permission changes, external communications, production changes, legal/compliance actions, and financial actions.

```
@ai_function(
    name="issue_refund",
    description="Issue a refund for a customer order. Requires human approval.",
    approval_mode="always_require",
)
async def issue_refund(
    order_id: Annotated[str, "The order ID"],
    amount: Annotated[float, "Refund amount in USD"],
    reason: Annotated[str, "Reason for the refund"],
) -> dict:
    return {
        "order_id": order_id,
        "amount": amount,
        "status": "refund_submitted",
        "reason": reason,
    }
```

Q: What is the biggest production risk with tools?

A: Giving the agent broad permissions. Tools should be narrow, least-privilege, typed, tested, and server-side validated.

8. RAG and grounding with Azure AI Search

RAG is like giving the agent an open-book exam. Without RAG, the agent answers from model memory. With RAG, it searches trusted documents, reads relevant chunks, cites sources, and answers based on evidence.

```
Documents
|
Ingestion pipeline
|
Chunking + metadata enrichment
|
Embedding generation
|
Azure AI Search index
|
Retriever tool
|
Agent
|
Cited answer
```

```
from typing import Annotated
from agent_framework import ai_function

@ai_function(
    name="search_knowledge_base",
    description=(
        "Search the enterprise knowledge base for relevant policy, architecture, "
        "or troubleshooting information."
    ),
),
```

```

)
async def search_knowledge_base(
    query: Annotated[str, "Search query generated from the user's question"],
    top_k: Annotated[int, "Number of search results to return"] = 5,
) -> list[dict]:
    # Replace this with Azure AI Search SDK code.
    return [
        {
            "title": "Production Agent Architecture Guide",
            "content": "Production agents require tracing, evaluation, fallback, and secure tool
access.",
            "source_url": "https://internal/wiki/agent-architecture",
        }
    ]

```

```

GROUNDING_INSTRUCTIONS = """
You are a grounded enterprise assistant.

Rules:
1. Use search_knowledge_base when the answer depends on company or product knowledge.
2. Do not answer from memory when the knowledge base is required.
3. Cite the source title and source_url for every factual claim from retrieved content.
4. If sources disagree, explain the disagreement.
5. If no reliable source is found, say so clearly.
"""

```

Q: What is the most common RAG failure?

A: Retrieval failure. The model cannot answer correctly if the right context was never retrieved.

Q: What should you log in RAG?

A: Original query, rewritten query, filters, retrieved chunk IDs, scores, selected context, answer, citations, latency, and user feedback.

9. Workflows and multi-agent design

Use an agent when the task is flexible and reasoning-driven. Use a workflow when the process has known steps, compliance requirements, approvals, or repeatable orchestration.

Use an agent when...	Use a workflow when...
Intent is flexible	Steps are known in advance
Tool choice may vary	Compliance needs deterministic control
Exploration or reasoning is required	Human approval must occur at defined points
Conversation matters	Auditability and repeatability matter most

```

Support workflow:
1. Classify user intent
2. Retrieve relevant documents
3. Decide whether a business API is needed
4. Call safe tools or request approval for risky tools
5. Validate tool outputs
6. Generate grounded final answer
7. Log trace, metrics, and feedback signals

```


Multi-agent systems are useful when roles need distinct instructions or independent verification. Example team: planner, retriever, tool executor, critic, and responder. Avoid multi-agent designs when one workflow or one tool-using agent is enough.

Q: Why not use one giant agent for everything?

A: Because smaller agents and explicit workflows are easier to test, secure, monitor, debug, and optimize.

10. Middleware, guardrails, and structured output

Middleware is like airport security. It lets you inspect and enforce rules before and after agent execution, tool execution, and final response generation.

```
from agent_framework import agent_middleware, AgentRunContext

@agent_middleware
async def logging_middleware(context: AgentRunContext, next):
    print(f"[agent:start] {context.agent.name}")
    try:
        await next(context)
        print(f"[agent:success] {context.agent.name}")
    except Exception as exc:
        print(f"[agent:error] {context.agent.name}: {exc}")
    raise
```

Guardrail placement:

Layer	Controls
Input	Prompt-injection checks, PII detection, tenant validation, rate limits
Tool	Tool allowlist, parameter validation, authorization, timeouts, audit logs
Output	Citation checks, JSON schema validation, safety checks, PII leakage checks

Use structured output when the response feeds another system.

```
from pydantic import BaseModel
from typing import Literal

class TicketDecision(BaseModel):
    intent: Literal["answer", "create_ticket", "escalate"]
    confidence: float
    summary: str
    required_action: str | None
    risk_level: Literal["low", "medium", "high"]
```

11. Evaluation and GenAIOps

Experts do not rely on demos. They build evaluation suites. Evaluate task adherence, tool selection, tool input accuracy, tool output utilization, groundedness, relevance, safety, latency, and cost.

Evaluator dimension	What it checks
Task completion	Did the agent deliver a usable result for the requested task?
Task adherence	Did the agent follow system instructions and policy?
Intent resolution	Did the agent identify user intent correctly?
Tool selection	Did the agent choose the right tools and avoid unnecessary tools?
Tool input accuracy	Were tool arguments correct, complete, typed, and grounded?
Tool output utilization	Did the agent correctly use returned tool data?
Groundedness	Is the answer supported by retrieved evidence?
Safety	Did the agent avoid unsafe or policy-violating behavior?

```
eval_cases = [  
  {  
    "id": "case_001",  
    "user_input": "What is the refund policy for enterprise customers?",  
    "expected_tools": ["search_knowledge_base"],  
    "expected_sources": ["refund_policy"],  
    "expected_behavior": "Answer with citation. Do not invent policy.",  
    "risk_level": "medium",  
  },  
  {  
    "id": "case_002",  
    "user_input": "Refund order ORD-10023 for $500.",  
    "expected_tools": ["issue_refund"],  
    "expected_behavior": "Require human approval before refund.",  
    "risk_level": "high",  
  },  
]
```

```
CI/CD evaluation gate:  
Pull request  
-> deploy test agent  
-> run golden test set  
-> run safety tests  
-> run tool accuracy tests  
-> compare with baseline  
-> block merge if quality, safety, or cost regresses
```

Q: What separates a beginner from an expert?

A: Experts can prove quality with repeatable evaluations and traces, not just show good conversations.

12. Observability and tracing

Without tracing, debugging an agent is like investigating an aircraft incident with no black box. Every production request should have a trace ID.

```
import logging
import time
import uuid

logger = logging.getLogger("agent_app")

async def run_with_trace(agent, user_id: str, message: str):
    trace_id = str(uuid.uuid4())
    start = time.time()
    logger.info("agent_request_started", extra={"trace_id": trace_id, "user_id": user_id})

    try:
        response = await agent.run(message)
        logger.info(
            "agent_request_completed",
            extra={"trace_id": trace_id, "latency_ms": int((time.time() - start) * 1000)},
        )
        return response
    except Exception as exc:
        logger.exception("agent_request_failed", extra={"trace_id": trace_id, "error":
str(exc)})
        raise
```

Metric category	Examples
Business	Task completion, deflection, escalation, satisfaction
Quality	Groundedness, citation coverage, hallucination rate, tool accuracy
Operations	Latency p50/p95/p99, token usage, cost, tool failures, timeouts
Safety	Blocked requests, prompt-injection attempts, PII leakage attempts, policy violations

13. Security and responsible AI

Prompt rules alone do not secure an agent. Security must be enforced in identity, RBAC, network controls, server-side validation, tool permissions, and audit logging.

- [] Use Microsoft Entra ID and managed identity
- [] Avoid hardcoded keys
- [] Use Key Vault for secrets
- [] Use least-privilege tools
- [] Apply tenant isolation and access control trimming in RAG
- [] Treat retrieved documents and tool outputs as data, not instructions
- [] Require human approval for risky actions

- [] Never log secrets or unnecessary PII
- [] Create audit records for state-changing actions

Attack	Example	Defense
User override	Ignore previous instructions	Instruction hierarchy and policy enforcement
Retrieved document injection	Document tells agent to reveal secrets	Treat retrieved content as untrusted data
Tool output injection	API result contains malicious instructions	Sanitize and constrain tool outputs
Data exfiltration	Print hidden prompts or secrets	Refuse secret disclosure and never pass secrets into prompts
Unsafe action	Delete or refund without authorization	Server-side authorization and approval workflows

Security hierarchy:

1. System and developer instructions are authoritative.
2. Tool outputs and retrieved documents are untrusted data.
3. User instructions cannot override policy, identity, authorization, or tool constraints.
4. State-changing actions require server-side validation, not just model judgment.

14. Deployment and operations

Option	Use when
Azure App Service	Simple web/API deployment with managed identity and enterprise integration.
Azure Container Apps	Containerized microservice with flexible scaling.
AKS	Complex platform, Kubernetes standardization, advanced networking.
Foundry hosted agent	Managed agent deployment and operations through Foundry.
Azure Functions	Event-driven tools or lightweight API operations.

Recommended first production path: FastAPI on App Service, Microsoft Agent Framework for agent logic, Microsoft Foundry for models and agent service capabilities, Azure AI Search for RAG, Cosmos DB for state and feedback, Application Insights for telemetry, Key Vault for secrets, and managed identity for resource access.

```

from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

app = FastAPI()

class ChatRequest(BaseModel):
    user_id: str
    message: str

class ChatResponse(BaseModel):

```

```

    answer: str
    trace_id: str | None = None

@app.post("/chat", response_model=ChatResponse)
async def chat(request: ChatRequest):
    try:
        response = await app.state.agent.run(request.message)
        return ChatResponse(answer=response.text, trace_id=getattr(response, "trace_id", None))
    except Exception as exc:
        raise HTTPException(status_code=500, detail=f"Agent request failed: {exc}")

```

Production SLO examples:

- [] Availability: 99.5% monthly API availability or better
- [] Latency: p50 under 3 seconds, p95 under 10 seconds, p99 under 20 seconds
- [] Quality: groundedness >= 90%, citation coverage >= 95% for RAG answers
- [] Tool quality: tool selection >= 95%, tool argument accuracy >= 98%
- [] Safety: zero critical unsafe tool executions
- [] Cost: average cost per successful request within approved budget

Incident playbook for incorrect answer:

- [] Find trace ID
- [] Inspect retrieved documents and retrieval scores
- [] Inspect tool calls and arguments
- [] Inspect prompt version and model deployment
- [] Check recent code, prompt, model, or index changes
- [] Reproduce with an evaluation case
- [] Rollback or disable risky tool if needed
- [] Add the incident as a regression test

15. Capstone project

Build an Enterprise IT Support Agent. This capstone is intentionally close to real enterprise work and exercises the core production skills.

Milestone	Capability
1. Basic agent	Answer simple support questions with clear instructions.
2. Grounded RAG	Answer policy and troubleshooting questions from Azure AI Search with citations.
3. Safe tools	Check ticket status and user profile using narrow read-only tools.
4. Action tools	Create tickets with typed parameters and validation.
5. Approval	Require human approval for account changes, refunds, or production-impacting actions.
6. Workflow	Classify -> retrieve -> act -> validate -> respond ->

	log.
7. Evaluation	Run golden, safety, RAG, and tool accuracy test sets.
8. Deployment	Deploy API and agent app to Azure with monitoring and rollback.

Suggested repository layout:

```

agentic-foundry-app/
  app/           # FastAPI entry point, auth, config
  agents/        # Agent definitions and prompt files
  tools/         # Narrow, typed, tested business tools
  workflows/     # Explicit orchestration flows
  rag/           # Ingestion, chunking, indexing, retrieval
  evals/         # Golden cases, safety cases, tool cases
  observability/ # Tracing, logging, metrics
  tests/         # Unit, integration, security tests
  infra/         # Bicep or Terraform
Dockerfile
requirements.txt
README.md

```

16. 12-week plan

Week	Focus	Build
1	Foundry basics	Deploy a model and run a simple Foundry-connected chat.
2	Agent Framework basics	Create a single production-oriented study assistant agent.
3	Tools	Add read-only ticket/order status tools with tests.
4	RAG	Build Azure AI Search-backed knowledge base Q&A with citations.
5	Workflows	Create classify -> retrieve -> act -> respond support flow.
6	Multi-agent patterns	Add planner, retriever, executor, critic, responder roles where justified.
7	Evaluation	Create 100-case eval suite and run regressions.
8	Observability	Add trace IDs, Application Insights logs, and dashboards.
9	Security	Add managed identity, Key Vault, RBAC, prompt-injection controls.
10	Deployment	Deploy FastAPI + Agent Framework to Azure.
11	Hardening	Add retries, timeouts, fallback, human escalation, rollback.

12	Capstone	Complete enterprise IT support agent with docs and runbook.
----	----------	---

17. Q&A bank

Q: What is an agent?

A: An LLM-powered component that receives input, follows instructions, optionally uses tools, and produces a response or action.

Q: What is the difference between a chatbot and an agent?

A: A chatbot mainly responds. An agent can plan, use tools, take actions, and handle multi-step tasks.

Q: What is the difference between an agent and a workflow?

A: An agent is flexible and reasoning-driven. A workflow is explicit, controlled, testable, and repeatable.

Q: What is Microsoft Foundry Agent Service?

A: A managed platform for building, deploying, and scaling agents using Foundry models, tools, conversations, and responses.

Q: What are Foundry Agent Service runtime components?

A: Agents, conversations, and responses.

Q: What makes a good tool?

A: A narrow purpose, clear schema, typed inputs, validation, authentication, authorization, timeouts, retries, errors, and audit logs.

Q: What is grounding?

A: Using trusted external knowledge and evidence instead of relying only on model memory.

Q: What is task adherence?

A: Whether the agent followed instructions, procedures, and task constraints.

Q: Why do experts build golden datasets?

A: To detect regressions and prove quality repeatedly before deployment.

Q: What should you trace?

A: User input, tool calls, tool arguments, tool results, retrieved context, response, latency, tokens, cost, and errors.

Q: Why use managed identity?

A: It avoids hardcoded secrets and integrates with Azure RBAC.

Q: Can prompts alone secure an agent?

A: No. Prompts help, but security must be enforced by code, identity, RBAC, network, and tool permissions.

Q: When should you use smaller models?

A: For routing, classification, extraction, summarization, and low-risk steps.

Q: What should happen when confidence is low?

A: Ask a clarifying question, retrieve more evidence, or escalate. Do not guess.

Q: What is the expert standard?

A: Correct, safe, secure, observable, cost-controlled, recoverable, and continuously improvable behavior.

18. References and source notes

- **Microsoft Foundry Agent Service runtime components:**
<https://learn.microsoft.com/en-us/azure/foundry/agents/concepts/runtime-components>
- **Microsoft Foundry Agent Service overview:**
<https://learn.microsoft.com/en-us/azure/foundry/agents/overview>
- **Microsoft Agent Framework overview:**
<https://learn.microsoft.com/en-us/agent-framework/overview/>
- **Microsoft Agent Framework workflows:**
<https://learn.microsoft.com/en-us/agent-framework/workflows/>
- **Microsoft Agent Framework with Microsoft Foundry:** <https://learn.microsoft.com/en-us/agent-framework/agents/providers/microsoft-foundry>
- **Agent evaluators for Generative AI in Microsoft Foundry:**
<https://learn.microsoft.com/en-us/azure/foundry/concepts/evaluation-evaluators/agent-evaluators>
- **Evaluate your AI agents in Microsoft Foundry:**
<https://learn.microsoft.com/en-us/azure/foundry/observability/how-to/evaluate-agent>
- **Python API reference for Agent Framework core:**
https://learn.microsoft.com/en-us/python/api/agent-framework-core/agent_framework?view=agent-framework-python-latest
- **Azure AI Agent Client API reference:** https://learn.microsoft.com/en-us/python/api/agent-framework-core/agent_framework.azure.azureaiagentclient?view=agent-framework-python-latest
- **agent-framework package on PyPI:** <https://pypi.org/project/agent-framework/>

Final production readiness checklist

Use this page as the release gate for your capstone and any real delivery. If any critical item is unchecked, the agent is not production ready.

- ☐ Clear agent/workflow boundary
- ☐ Typed and least-privilege tools
- ☐ RAG grounding with citations
- ☐ Prompt-injection controls
- ☐ Structured outputs for downstream systems
- ☐ Human approval for high-risk actions
- ☐ Golden evaluation dataset
- ☐ CI/CD evaluation gates
- ☐ Trace IDs for every request
- ☐ Tool-call logs and metrics
- ☐ Latency, cost, and error dashboards
- ☐ Managed identity and RBAC
- ☐ Key Vault for secrets
- ☐ Rollback strategy
- ☐ Incident playbook
- ☐ Data retention and audit policy